# SIMATS ENGINEERING

## ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

**COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS** **COURSE CODE: MLA01**

---

### COURSE OUTCOME COVERED

**CO 5:** *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

---

**Total Marks: 100** **Time: 90 Mins**

**No. of Questions: 1**

Annexure A

### Modelling Comparative Analysis Assignment

---

***Code of Conduct:***

*I S.Venkateswarlu/192425340 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** **S.Venkateswarlu**

| Criterion | Weightage (%) | Marks Obtained |
|---|---|---|
| 1. Problem Analysis and Domain Understanding | 10% | |
| 2. Domain Knowledge and Fact Identification | 10% | |
| 3. Production Rule Modelling | 15% | |
| 4. Propositional Logic Modelling | 10% | |
| 5. First-Order Logic Modelling | 15% | |
| 6. Prolog Modelling and Implementation | 15% | |
| 7. Forward and Backward Chaining Demonstration | 10% | |
| 8. Comparative Analysis of Modelling Approaches | 10% | |
| 9. Results, Discussion and Model Selection | 3% | |
| 10. Documentation, GitHub Repository and Presentation | 2% | |
| **Total** | **100%** | |

Signature of the Faculty Total Marks Awarded

# AUTOMOBILE FAULT DIAGNOSIS USING PRODUCTION RULES, LOGIC AND PROLOG

## 1. Problem Statement & Objectives

### Problem Statement

An automobile service center receives vehicles with different complaints such as engine overheating, starting failure, abnormal engine noise, low mileage, and warning-light activation. Identifying the probable cause of these symptoms requires automobile knowledge and logical reasoning.

The objective of this project is to model the automobile fault diagnosis problem using **Production Rules, Propositional Logic, First-Order Logic, and Prolog**. The different approaches are compared based on expressiveness, inference mechanism, complexity, scalability, flexibility, and explainability.

### Objectives

1. To analyze the automobile fault diagnosis problem.
2. To identify important automobile facts and relationships.
3. To represent automobile knowledge using production rules.
4. To model the problem using propositional logic.
5. To model the problem using first-order logic.
6. To implement the knowledge base using Prolog.
7. To demonstrate forward chaining.
8. To demonstrate backward chaining.
9. To compare the four modelling approaches.
10. To select the most suitable model for automobile fault diagnosis.

### Input

The system considers symptoms such as:

- Engine overheating
- Low coolant
- Difficulty starting
- Weak battery
- Abnormal engine noise
- Low engine oil
- Low mileage

- Poor acceleration
- Warning indicator
- Electrical problem

**Output**

The system produces:

- Probable vehicle fault
- Supporting conditions
- Recommended diagnostic action

**2. Domain Knowledge & Fact Identification**

The selected domain is **Automobile Fault Diagnosis**.

A vehicle contains several systems, including the cooling system, battery system, ignition system, lubrication system, fuel system, and electrical system. Failure in these systems can produce observable symptoms.

| **Entity** | **Description** |
|---|---|
| Vehicle | Vehicle being diagnosed |
| Symptom | Observable vehicle condition |
| Fault | Possible cause of the problem |
| System | Vehicle subsystem |
| Action | Recommended diagnostic activity |

**Facts**

The following facts are considered:

Car1 has engine overheating.

Car1 has low coolant.

Car2 has difficulty starting.

Car2 has weak battery.

Car3 has abnormal engine noise.

Car3 has low engine oil.

Car4 has low mileage.

Car4 has poor acceleration.

Car5 has warning indicator.

Car5 has electrical problem.

**Relationships**

Vehicle → has → Symptom

Symptom + Symptom → indicates → Fault

Fault → belongs to → Vehicle System

Fault → requires → Diagnostic Action

**3. Production Rule Model**

Production rules represent knowledge using **IF–THEN** statements.

**Rule 1**

IF engine overheating

AND low coolant

THEN cooling system problem.

**Rule 2**

IF difficulty starting

AND weak battery

THEN battery problem.

**Rule 3**

IF abnormal engine noise

AND low engine oil

THEN lubrication problem.

**Rule 4**

IF low mileage

AND poor acceleration

THEN fuel system problem.

**Rule 5**

IF warning indicator

AND electrical problem

THEN electrical system problem.

**Recommendation Rules**

IF cooling system problem

THEN check coolant and radiator.

IF battery problem

THEN test battery and charging system.

IF lubrication problem

THEN check engine oil.

IF fuel system problem

THEN inspect fuel system and air filter.

IF electrical system problem

THEN inspect wiring and sensors.

**Example**

Given:

engine overheating

low coolant

Apply:

IF engine overheating AND low coolant

THEN cooling system problem

Therefore:

Cooling system problem

Then:

IF cooling system problem

THEN check coolant and radiator

Final recommendation:

Check coolant and radiator.

**4. Propositional Logic Model**

Propositional logic represents statements as either **True or False**.

**Propositions**

Let:

O = Engine overheating

C = Low coolant

S = Cooling system problem

D = Difficulty starting

B = Weak battery

BP = Battery problem

N = Abnormal engine noise

L = Low engine oil

LP = Lubrication problem

M = Low mileage

A = Poor acceleration

FP = Fuel system problem

W = Warning indicator

E = Electrical problem

EP = Electrical system problem

**Logical Rules**

Cooling system:

$O \land C \rightarrow S$

Battery:

$D \land B \rightarrow BP$

Lubrication:

$N \land L \rightarrow LP$

Fuel system:

$M \land A \rightarrow FP$

Electrical system:

$W \land E \rightarrow EP$

**Example Inference**

Given:

O = True

C = True

Using:

$O \land C \rightarrow S$

We obtain:

S = True

Therefore:

Cooling system problem = True

**Limitation**

Propositional logic is simple and easy to understand, but it cannot naturally represent individual vehicles and complex relationships.

**5. First-Order Logic Model**

First-Order Logic is more expressive than propositional logic because it supports **variables, predicates, objects, relationships, and quantifiers**.

**Predicates**

Symptom(Vehicle, Condition)

Fault(Vehicle, Problem)

Recommendation(Problem, Action)

For example:

Symptom(Car1, EngineOverheating)

Symptom(Car1, LowCoolant)

**Rule 1**

$\forall x$ [Symptom(x, EngineOverheating)

$\land$ Symptom(x, LowCoolant)

$\rightarrow$ Fault(x, CoolingSystemProblem)]

Meaning:

For every vehicle x, if the vehicle has engine overheating and low coolant, then the vehicle has a cooling system problem.

**Rule 2**

∀x [Symptom(x, DifficultyStarting)

∧ Symptom(x, WeakBattery)

→ Fault(x, BatteryProblem)]

**Rule 3**

∀x [Symptom(x, AbnormalNoise)

∧ Symptom(x, LowEngineOil)

→ Fault(x, LubricationProblem)]

**Rule 4**

∀x [Symptom(x, LowMileage)

∧ Symptom(x, PoorAcceleration)

→ Fault(x, FuelSystemProblem)]

**Rule 5**

∀x [Symptom(x, WarningIndicator)

∧ Symptom(x, ElectricalProblem)

→ Fault(x, ElectricalSystemProblem)]

**Example**

Facts:

Symptom(Car1, EngineOverheating)

Symptom(Car1, LowCoolant)

Rule:

∀x [Symptom(x, EngineOverheating)

∧ Symptom(x, LowCoolant)

→ Fault(x, CoolingSystemProblem)]

Substitute:

x = Car1

Therefore:

Fault(Car1, CoolingSystemProblem)

**6. Prolog Model & Implementation**

**Prolog Facts**

```
symptom(car1, engine_overheating).
symptom(car1, low_coolant).
```

```
symptom(car2, difficulty_starting).
symptom(car2, weak_battery).

symptom(car3, abnormal_noise).
symptom(car3, low_engine_oil).

symptom(car4, low_mileage).
symptom(car4, poor_acceleration).

symptom(car5, warning_indicator).
symptom(car5, electrical_problem).
```

**Prolog Rules**

```
fault(Vehicle, cooling_system_problem) :-
    symptom(Vehicle, engine_overheating),
    symptom(Vehicle, low_coolant).

fault(Vehicle, battery_problem) :-
    symptom(Vehicle, difficulty_starting),
    symptom(Vehicle, weak_battery).

fault(Vehicle, lubrication_problem) :-
    symptom(Vehicle, abnormal_noise),
    symptom(Vehicle, low_engine_oil).

fault(Vehicle, fuel_system_problem) :-
    symptom(Vehicle, low_mileage),
    symptom(Vehicle, poor_acceleration).

fault(Vehicle, electrical_system_problem) :-
    symptom(Vehicle, warning_indicator),
    symptom(Vehicle, electrical_problem).
```

**Recommendation Rules**

```
recommendation(cooling_system_problem,
```

```
        check_coolant_and_radiator).

recommendation(battery_problem,
        test_battery_and_charging_system).

recommendation(lubrication_problem,
        check_engine_oil).

recommendation(fuel_system_problem,
        inspect_fuel_system_and_air_filter).

recommendation(electrical_system_problem,
        inspect_wiring_and_sensors).
```

**Diagnosis Predicate**

```
diagnose(Vehicle, Fault, Action) :-
   fault(Vehicle, Fault),
   recommendation(Fault, Action).
```

**Query**

```
?- diagnose(car1, Fault, Action).
```

**Output**

Fault = cooling_system_problem,
Action = check_coolant_and_radiator.

**7. Forward & Backward Chaining Demonstration**

**Forward Chaining**

Forward chaining starts with known facts and moves toward a conclusion.

**Initial Facts**

Engine overheating

Low coolant

**Apply Rule**

Engine overheating + Low coolant

↓

Cooling system problem

**Apply Recommendation Rule**

Cooling system problem

↓

Check coolant and radiator

**Inference Sequence**

Fact 1: Engine overheating

Fact 2: Low coolant

↓

Production Rule

↓

Cooling system problem

↓

Recommendation Rule

↓

Check coolant and radiator

**Backward Chaining**

Backward chaining starts with a goal and works backward to find supporting facts.

**Goal**

Cooling system problem

Find the rule:

IF engine overheating

AND low coolant

THEN cooling system problem

Now check:

Is engine overheating true?

YES

Is low coolant true?

YES

Therefore:

Cooling system problem = TRUE

**Inference Sequence**

Goal:

Cooling system problem

↓

Find matching rule

↓

Check engine overheating

↓

Check low coolant

↓

Both conditions satisfied

↓

Goal proven

**8. Comparative Analysis of Modelling Approaches**

| **Parameter** | **Production Rules** | **Propositional Logic** | **First-Order Logic** | **Prolog** |
|---|---|---|---|---|
| Knowledge Representation | IF–THEN rules | True/False propositions | Predicates and relationships | Facts and rules |
| Expressiveness | Medium | Low | Very High | Very High |
| Rule Representation | Very easy | Logical implication | Logical formulas | Natural rule syntax |
| Inference Mechanism | Rule matching | Logical inference | Formal inference | Unification and backtracking |
| Forward Chaining | Easy | Possible | Possible | Can be implemented |
| Backward Chaining | Possible | Possible | Possible | Naturally supported |
| Handling Relationships | Limited | Poor | Excellent | Excellent |
| Scalability | Medium | Low–Medium | High | High |
| Explainability | High | High | High | High |
| Ease of Implementation | Easy | Easy | Difficult | Easy–Medium |

| Parameter | Production Rules | Propositional Logic | First-Order Logic | Prolog |
|---|---|---|---|---|
| Suitability | Good | Moderate | Very Good | Excellent |

**Comparison Summary**

**Production Rules:** Easy to understand and suitable for simple diagnostic problems.

**Propositional Logic:** Mathematically simple but cannot easily represent different vehicles and their relationships.

**First-Order Logic:** Highly expressive and capable of representing objects, variables, and relationships.

**Prolog:** Combines logical representation with executable inference and supports unification, backtracking, and query-based reasoning.

## 9. Results, Discussion & Model Selection

The four modelling approaches successfully represent the automobile fault diagnosis problem. Production rules provide a straightforward way of representing troubleshooting knowledge. Propositional logic provides simple logical reasoning but has limited representation capability. First-order logic provides better representation because it can represent vehicles, symptoms, relationships, and variables.

Prolog provides an executable implementation of logical knowledge. It automatically performs unification, matching, and backtracking.

**Model Selection**

**Prolog is selected as the most suitable model for this problem.**

The main reasons are:

1. It represents facts and rules clearly.
2. It supports variables.
3. It supports relationships between objects.
4. It performs automatic logical inference.
5. It supports backward chaining naturally.
6. It supports unification.
7. It supports backtracking.
8. It is easy to modify by adding new rules.
9. It provides explainable conclusions.
10. It is suitable for expert-system development.

## 10. Results, Limitations & Future Enhancements

### Test Cases

| Test Case | Vehicle | Symptoms | Expected Fault | Recommended Action |
|---|---|---|---|---|
| 1 | Car1 | Overheating + Low coolant | Cooling system problem | Check coolant and radiator |
| 2 | Car2 | Starting failure + Weak battery | Battery problem | Test battery |
| 3 | Car3 | Abnormal noise + Low oil | Lubrication problem | Check engine oil |
| 4 | Car4 | Low mileage + Poor acceleration | Fuel system problem | Inspect fuel system |
| 5 | Car5 | Warning indicator + Electrical problem | Electrical system problem | Inspect wiring and sensors |

### Limitations

1. The system depends on predefined rules.
2. It cannot automatically learn new faults.
3. It cannot physically inspect vehicle components.
4. Some vehicle faults can have similar symptoms.
5. The system provides probable faults rather than guaranteed diagnoses.
6. The current knowledge base contains only a limited number of symptoms.

### Future Enhancements

1. Add more vehicle faults and symptoms.
2. Support different vehicle models.
3. Add real-time vehicle sensor data.
4. Integrate OBD diagnostic information.
5. Add confidence scores.
6. Develop a graphical user interface.
7. Add natural-language input.
8. Combine Prolog with machine-learning techniques.
9. Create a larger automobile knowledge base.
10. Provide detailed explanations for each diagnosis.

## 11. Conclusion

The **Automobile Fault Diagnosis** problem was modelled using Production Rules, Propositional Logic, First-Order Logic, and Prolog.

Production rules provide a simple IF–THEN representation. Propositional logic provides basic logical reasoning. First-order logic provides a more expressive representation using predicates, variables, and relationships. Prolog converts the logical representation into an executable expert system.

Forward chaining was demonstrated by starting with known symptoms and deriving a fault. Backward chaining was demonstrated by starting with a suspected fault and checking whether the required symptoms were available.

Based on expressiveness, inference capability, flexibility, explainability, and implementation suitability, **Prolog is selected as the most appropriate approach for the automobile fault diagnosis system**.

## 12. References


1. Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
2. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
3. SWI-Prolog Documentation.
4. Automobile maintenance and troubleshooting manuals.
5. Standard vehicle service and diagnostic documentation.


## GitHub Repository

automobile-fault-diagnosis/

```
│
├── automobile_diagnosis.pl
├── production_rules.txt
├── propositional_logic.txt
├── first_order_logic.txt
├── test_cases.txt
├── README.md
└── screenshots/
    ├── test1.png
    ├── test2.png
    ├── test3.png
    ├── test4.png
    └── test5.png
```